

# Обробка винятків

Якісна програма повинна передбачати можливість виникнення помилок виконання і правильно реагувати на них, а не завершуватись аварійно зі стандартними повідомленнями про помилку. Для реалізації цього в Python вбудований механізм обробки винятків (**exception handling**), який ґрунтується на тому, що коли під час виконання синтаксично коректної програми стається помилка, інтерпретатор генерує виняток. Ці винятки, які викликають збій програми, можна контролювати. Крім контролю за помилками, обробка винятків використовується для гарантії виконання певних заключних операцій і для повідомлення про настання певних подій.

## Основні типи винятків

Інтерпретатор Python підтримує велику кількість вбудованих типів винятків, зокрема:

- `FileNotFoundError` (виникає при спробі відкриття неіснуючого файлу);
- `ImportError` (при спробі імпорту неіснуючого модуля);
- `IndexError` (при спробі доступу до елемента з неіснуючим індексом);
- `KeyError` (при спробі доступу до елемента словника з неіснуючим ключем);
- `NameError` (при спробі доступу до неіснуючого об'єкту);
- `TypeError` (при спробі виконати недоступну операцію);
- `ZeroDivisionError` (при спробі виконання ділення на 0).

```
>>> print(55/0)
```

```
Traceback (most recent call last):
```

```
  File "<interactive input>", line 1, in <module>
```

```
ZeroDivisionError: integer division or modulo by zero
```

```
>>> a = []
>>> print(a[5])
```

```
Traceback (most recent call last):
```

```
File "<interactive input>", line 1, in <module>
```

```
IndexError: list index out of range
```

```
>>> tup = ("a", "b", "d", "d")
```

```
>>> tup[2] = "c"
```

```
Traceback (most recent call last):
```

```
File "<interactive input>", line 1, in <module>
```

```
TypeError: 'tuple' object does not support item assignment
```

## Інструкція try

Для обробки винятків використовується інструкція `try`, найпростіший синтаксис якої складається з двох блоків: власне `try` і блоку `except`:

```
try:
    # цей блок виконується завжди
    ...
except:
    # цей блок виконується, якщо у блоці try трапиться виняток
    ...
```

Тобто, код, під час виконання якого можуть трапитися помилки виконання (винятки), вкладається в блок `try`, а блок `except` містить код, який виконуватиметься тільки коли в блоці `try` трапиться виняток (неважливо якого типу). Блок `except` запобігає аварійному завершенню програми, а після його виконання (або невиконання в разі успішного виконання блоку `try`) керування завжди передається наступній (після конструкції `try/except`) інструкції.

У блоці `except` можна вказувати конкретний тип винятку, крім того, таких блоків може бути декілька:

```

try:
    # цей блок виконується завжди
    ...
except <ТипВинятку1>:
    # цей блок виконується, якщо у блоці try
    # трапиться виняток першого типу
    ...
except <ТипВинятку2>:
    # цей блок виконується, якщо у блоці try
    # трапиться виняток другого типу
    ...
...
except:
    # цей необов'язковий блок виконується,
    # якщо у блоці try трапиться виняток
    # будь-якого типу, відмінного від перерахованих
    # у попередніх блоках except
    ...

```

Крім блоків `except`, інструкція `try` може мати ще два необов'язкові блоки: `else` і `finally`. Перший виконується тільки якщо блок `try` виконався без винятків, а другий — завжди, чим гарантує виконання принципово важливих заключних операцій незалежно від того, трапилися винятки чи ні.

Отже, повна форма інструкції `try` має такий вигляд:

```

try:
    # цей блок виконується завжди
    ...
except <ТипВинятку1>:
    # цей блок виконується, якщо у блоці try
    # трапиться виняток першого типу
    ...
except <ТипВинятку2>:
    # цей блок виконується, якщо у блоці try
    # трапиться виняток другого типу
    ...
...
except:
    # цей необов'язковий блок виконується, якщо у блоці try
    # трапиться виняток будь-якого типу, відмінного
    # від перерахованих у попередніх блоках except
    ...
else:

```

```

    # цей необов'язковий блок виконується, тільки якщо блок try
    # виконався без винятків
    ...
finally:
    # цей необов'язковий блок виконується завжди, гарантуючи
    # виконання важливих заключних операцій незалежно від того,
    # трапилися винятки чи ні
    ...

```

**Деякі особливі часткові випадки:**

1)

```

try:
    ...
except <ТипВинятку1>:
    ...
except <ТипВинятку2>:
    ...
else:
    # блок виконається, тільки якщо не трапиться жодного винятку -
    # він не виконається, якщо трапиться інший (не передбачений в
    # блоках except) виняток - програма завершиться аварійно!
    ...

```

2)

```

try:
    ...
finally:
    # якщо станеться виняток, блок виконається, але потім
    # програма аварійно завершиться
    ...

```

Розглянемо процес використання механізму обробки винятків на прикладі програми, яка обчислює суму чисел, записаних у деякому текстовому файлі по одному числу в рядку:

```

fileName = input("Введіть ім'я файлу: ")

with open(fileName) as f:      # відкриття файлу в режимі читання
    lines = f.readlines()      # формування списку рядків файлу

```

```
# перетворення списку рядків у список чисел
numbers = list(map(float, lines))

print(sum(numbers))          # виведення суми елементів списку
```

Ця програма передбачає принаймі дві речі:

- 1) файл з введеним іменем існує;
- 2) у кожному рядку файлу записане число.

Якщо хоча б один пункт не виконується, виконання програми призведе до аварійного завершення з повідомленням про відповідну помилку:

- 1) `FileNotFoundError: [Errno 2] No such file or directory.`
- 2) `ValueError: could not convert string to float.`

Тому для коректного виконання програми інструкції, для яких існує ризик помилки, потрібно вкласти в інструкцію `try`:

```
fileName = input("Введіть ім'я файлу")

try:
    with open(fileName) as f:
        lines = f.readlines()
        numbers = list(map(float, lines))
except FileNotFoundError:
    print('Файлу з вказаним іменем не знайдено!')
except ValueError:
    print('Файл містить нечислові дані!')
except:
    print('Щось пішло не так')
else:
    print(sum(numbers))
finally:
    print('Програма завершила роботу.')
```

## Використання обробки винятків у функціях

```
def is_file_exists(filename):
    try:
        f = open(filename)
        f.close()
        return True
    except:
        return False
```

## Користувацькі винятки (інструкція raise)

```
age = int(input('Enter your age, please: '))
if age < 0 or age > 100:
    raise ValueError(age, 'is not a valid age')
print(age)
```

```
Please enter your age: -66
Traceback (most recent call last):
  File "/home/mokasin/1.py", line 4, in <module>
    raise my_error
ValueError: (-66, ' is not a valid age')
```

## Дотримання обмежень (інструкція assert)

```
age = int(input('Enter your age, please: '))
assert 0 < age <= 100, 'age must be between 1 and 100'
print(age)
```